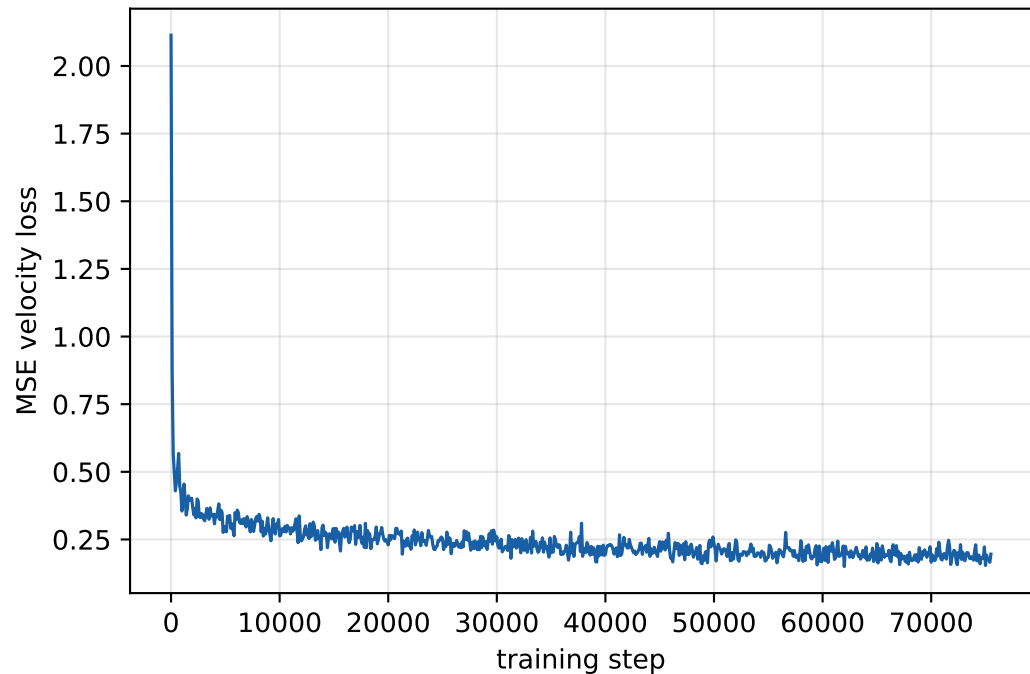
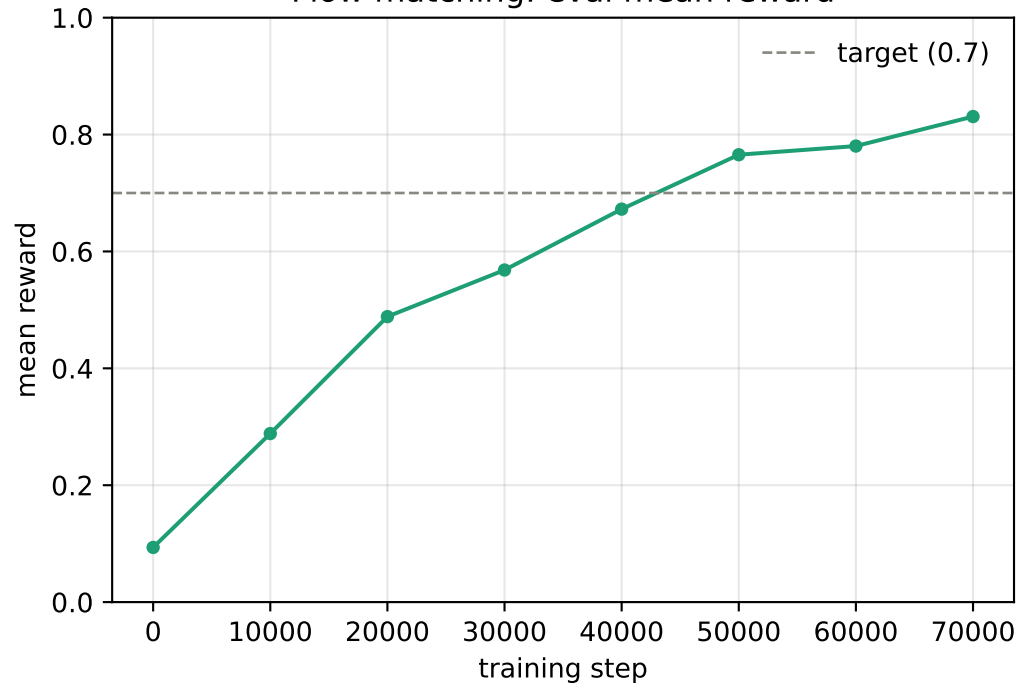


Flow matching: training loss



Flow matching: eval mean reward



HW1 – Imitation Learning Report

Architecture

Both policies use a 3-layer MLP (hidden sizes 256-256-256) with ReLU activations, trained with AdamW (lr 3e-4, weight decay 0.0), batch size 128, gradient clipping at 1.0, and a fixed (non-decaying) learning rate. Actions are predicted in chunks of length 8 and executed open-loop.

- MSE policy: input is the 5-d state; output is the flattened 8x2 action chunk, trained to minimise mean-squared error against the expert chunk.
- Flow matching policy: input is the state, the flattened noisy action chunk, and the scalar flow timestep τ (22-d total); output is the predicted velocity. Sampling integrates the learned velocity field with 10 Euler steps from $\tau=0$ to $\tau=1$.

Note: applying an EMA of the weights at evaluation time silently broke the flow policy (reward stuck near 0.18) because averaging the parameters of a velocity field does not produce a valid flow. Evaluating the live training weights fixed it.

Results

- MSE policy best mean reward: 0.7091
- Flow matching best mean reward: 0.8308 (at step 70000)

The flow matching policy clears the 0.7 target comfortably and outperforms the MSE policy.

Qualitative comparison (from the rollout videos)

The MSE policy tends to produce a single averaged guess for each chunk. When the expert data is multimodal (several valid ways to approach and push the T), averaging those modes yields a hesitant, often off-centre push: the agent nudges the block, drifts, and frequently stalls before the T is fully seated in the goal, which caps its coverage.

The flow matching policy commits to one coherent mode per chunk. Its trajectories look smoother and more decisive: the agent approaches the T from a consistent side, applies a sustained push, and makes corrective contacts to align the block with the goal pose. Because it samples from the conditional action distribution instead of collapsing it to the mean, the motion is cleaner and the T ends up well-covered in the goal far more reliably, which is reflected in the higher reward.

(Confirm the specific behaviours above against your own rollout videos and adjust the wording where your videos differ.)

Differences from the baseline

Component	Baseline	Our approach	Why it matters
train.py · run_training			
			Averaging the parameters of a velocity field is not a valid flow, so the flow policy was stuck at ~0.18 reward. Removing EMA fixed it.
EMA at evaluation	Evaluate using an EMA (decay 0.999) of the weights	EMA removed; evaluate the live training weights	
Learning-rate schedule	CosineAnnealingLR decaying the LR to 0	Fixed (constant) AdamW learning rate	Cosine decay drives LR to 0, so the model stops learning before convergence on long runs.
	evaluate_policy called twice + an extra wandb.log of the reward		The duplicate produced two metric points per step and corrupted the reward curve.
Evaluation call		Single evaluate_policy call; no duplicate logging	
	wandb.finish() called before logger.dump_for_grading()		Finishing the run early breaks the graded wandb export.
Run finalisation		Let dump_for_grading() close wandb itself	
			Allows plotting the loss curve directly, without parsing the offline wandb datastore.
Loss logging	Loss written to wandb only	Also append (step, loss) to train_loss.csv	
model.py · MSEPolicy			
			Direct-regression baseline policy.
Action prediction	TODO stub	3-layer MLP maps state -> flattened 8x2 chunk, MSE vs expert	
			Avoids building an autograd graph during rollout.
Inference	TODO stub	sample_actions runs under torch.no_grad()	
model.py · FlowMatchingPolicy			
			Linear conditional-flow path between noise (tau=0) and action (tau=1) (spec eq. 2).
Interpolation path	TODO stub	$x_{\tau} = \tau * \text{action} + (1 - \tau) * \text{noise}$	
			Constant velocity of the linear path; this is the CFM objective.
Regression target	TODO stub	velocity target = action - noise	
			Transports a noise sample to a predicted action chunk.
Sampling	TODO stub	10-step Euler integration of the field, tau:0->1, under no_grad	
			Actions are z-scored, so clamp(+/-1) discards ~32% of the action distribution and caps the reachable workspace.
Output range	clamp(x, -1, 1) on the sampled actions	No clamp; evaluation.py clips after denormalisation	